LITERATURE SUMMARY

From Code Pre-training to Agentic Unit Test Generation

CodeT5 (2021) · TestGen-LLM at Meta (2024) · TestAgent (2026)

Three papers, one trajectory

CodeT5 · 2021

Pre-train one encoder-decoder model that understands code structure, then fine-tune for many tasks.

TestGen-LLM · 2024

Use LLMs inside a filtering pipeline at Meta, so only verified test improvements are proposed.

TestAgent · 2026

Let cooperating LLM agents plan, write and review tests, using a repository knowledge graph.

Shift: better models, then verified outputs, then autonomous multi-agent workflows.

CodeT5: identifier-aware pre-training

Salesforce Research, EMNLP 2021. A T5-based model for both code understanding and generation, pre-trained on 8.35M code and comment instances.

Identifier-aware denoising

Masked span prediction, identifier tagging and masked identifier prediction teach the model which tokens are identifiers.

Bimodal dual generation

Code-to-text and text-to-code trained together to align natural language with code.

One unified model

Six tasks and 14 CodeXGLUE sub-tasks, with multi-task fine-tuning via task prefixes.

43.20

CodeBLEU on code generation (PLBART: 38.52)

13.96

Exact match, medium bug-fix refinement (prior best: 9.10)

65.78

Defect detection accuracy (PLBART: 63.18)

Wang, Wang, Joty, Hoi. CodeT5-base results.

TestGen-LLM: assured test improvement at Meta

Extends existing Kotlin test classes. Each LLM-generated test must clear three filters, which removes hallucinated or useless output.

1. Builds

Candidate must compile inside the real app build.

2. Passes reliably

Must pass on five runs, which rules out flaky tests.

3. Adds coverage

Must raise line coverage over all existing tests.

75% / 57% / 25%

of 86 classes: built / passed reliably / added coverage

10%

of 1,979 classes improved; 73% of changes accepted by engineers

Alshahwan et al., FSE 2024. Instagram and Facebook test-a-thons, late 2023.

TestAgent: multi-agent, human-inspired workflow

Mirrors how developers test: understand the method, write and run tests, review quality. Agents call tools over a test-specific knowledge graph.

Requirement planner

Summarizes the focal method and derives test requirements: normal, boundary, exceptional.

Test generator

Retrieves context, writes tests, runs them, and analyzes failures. Three LLMs vote on real bugs.

Test reviewer

Scores coverage and mutation results, then accepts, discards, updates or augments tests.

92.34%

line coverage on six Java projects (HITS: 82.39%)

83.69%

mutation score (EvoSuite: 43.59%)

154 / 167

suspected failures confirmed as real bugs

Zhang et al., 2026. GPT-4o backbone; Python port: 88.85% line coverage.

Side-by-side comparison

Aspect	CodeT5	TestGen-LLM	TestAgent
Contribution	Pre-trained model	Verified deployment pipeline	Multi-agent framework
Task	Many code tasks	Extend Kotlin tests	Java and Python unit tests
Key idea	Identifier awareness	Build, pass, coverage filters	Knowledge graph plus agents
Evidence	CodeXGLUE benchmarks	Industrial deployment	Benchmarks, industry, user study
Main limit	Pre-LLM scale, BLEU metrics	Line coverage only; low per-test yield	Single run; cost and time

Compiled from the three papers' reported results.

Takeaways and discussion

- Code structure (identifiers) improves pre-trained models.
- Automatic verification makes LLM output trustworthy in production.
- Repository-level context and review loops lift coverage and mutation scores.
- Open question: coverage and mutation score still proxy for test quality.
- Open question: results are single-run, and benchmarks may overlap LLM training data.